

SDEV 1001

Programming Fundamentals

Classes and Objects 4 - Inheritance

A LEADING POLYTECHNIC COMMITTED TO YOUR SUCCESS

Expectations - What I expect from you

- No Late Assignments

- No Cheating

- Be a good classmate

- Don’t waste your time

- Show up to class

WE ARE ESSENTIAL TO ALBERTA | NAIT

Agenda

On the right is what we will cover today.

- What is Inheritance?
- Why Use Inheritance?
- Parent and Child Classes
- Defining a Child Class
- Calling the Parent Constructor with super()
- Overriding Methods
- Using super() to Extend a Parent Method
- Adding Child-Only Methods
- Guarding with isinstance()
- Summary

WE ARE ESSENTIAL TO ALBERTA | NAIT

What is Inheritance?

Inheritance is a way for one class to take on the attributes and methods of another class without rewriting them.

- A **parent class** (also called a base class) defines the shared attributes and methods.
- A **child class** (also called a subclass) inherits from the parent and can add to or change its behaviour.

Analogy

Think of a family tree. Children inherit traits from their parents — they get all of those traits for free, but they can also have their own traits that make them unique.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Why Use Inheritance?

- **Code reuse**: Shared attributes and methods only need to be written once, in the parent.
- **Modeling "is-a" relationships**: A `Dog` *is an* `Animal`. Inheritance lets the code reflect that.
- **Overriding behaviour**: A child can replace a parent method when it needs to behave differently, without touching the parent.
- **Extensibility**: Adding a new child class doesn’t require changes to existing code.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Parent and Child Classes

On the right we have an `Animal` class. It defines the attributes and methods that every animal will share.

- Every animal has a `name`, a `color`, and a `num_legs`.
- Every animal can `eat`, `make_a_noise`, and `move`.

This will be our **parent class**. In the next slides we'll build child classes that inherit all of this for free.

```python
class Animal:
    def __init__(self, name: str, color: str, num_legs: int
        self.name = name
        self.color = color
        self.num_legs = num_legs

    def eat(self) -> None:
        print(f"{self.name} starts eating. Nom nom nom ... ")

    def make_a_noise(self) -> None:
        print(f"{self.name} makes an interesting noise.")

    def move(self) -> None:
        print(f"{self.name} runs around!")

    def __str__(self) -> str:
        return f"Name: {self.name}\nColor: {self.color}"
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

Defining a Child Class

To make a child class, put the parent's name in parentheses after the child's name.

```
class Dog(Animal):
    pass
```

That's it — `Dog` now has every attribute and method that `Animal` has, even though we haven't written any code inside `Dog` yet.

```
ranger = Dog("Ranger", "Brown", 4)
print(ranger)
ranger.eat()
ranger.make_a_noise()
```

Output:

```
Name: Ranger
Color: Brown
Ranger starts eating. Nom nom nom ...
Ranger makes an interesting noise.
```

`Dog` doesn't define `__init__`, `eat`, or `make_a_noise` itself — it inherits all of them from `Animal`.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Calling the Parent Constructor with super()

When a child needs its own attributes, give it its own `__init__`. Use `super().__init__(...)` to run the parent's constructor first, then set the child's extra attributes.

- `super()` is a reference to the **parent class**.
- Calling `super().__init__(...)` runs `Animal.__init__`, which sets `name`, `color`, and `num_legs`.
- After that, set the new attribute (`breed`).

```python
class Dog(Animal):
    def __init__(
        self,
        name: str,
        color: str,
        num_legs: int,
        breed: str,
    ) -> None:
        super().__init__(name, color, num_legs)
        self.breed = breed

ranger = Dog("Ranger", "Brown", 4, "Labrador")
print(ranger.name)   # Ranger
print(ranger.breed)  # Labrador
```

Without `super().__init__(...)`, the parent's attributes would never get set.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Overriding Methods

A child can **override** a parent method by defining a method with the same name. The child's version runs instead of the parent's.

- All animals make some noise, but a dog barks.
- Define `make_a_noise` in `Dog` and Python uses that one for `Dog` instances.
- The parent's `make_a_noise` is untouched — every other `Animal` still uses it.

```
class Dog(Animal):
    def __init__(self, name: str, color: str, num_legs: int
        super().__init__(name, color, num_legs)
        self.breed = breed

    def make_a_noise(self) -> None:
        print(f"{self.name} starts barking loudly! Bark Bar

ranger = Dog("Ranger", "Brown", 4, "Labrador")
ranger.make_a_noise()
```

Output:

```
Ranger starts barking loudly! Bark Bark!
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

Using super() to Extend a Parent Method

Sometimes you don't want to *replace* the parent's method — you want to **add to it**. Call `super().method()` from inside the override and use the result.

- `Animal.__str__` already returns the name, color, and number of legs.
- `Dog.__str__` calls `super().__str__()` to get that string, then appends the breed.
- No duplicated code, and if `Animal.__str__` ever changes, `Dog` picks the change up automatically.

```
class Dog(Animal):
    #  ... existing code ...

    def __str__(self) -> str:
        return_string = super().__str__()
        return_string += f"\nBreed: {self.breed}"
        return return_string

ranger = Dog("Ranger", "Brown", 4, "Labrador")
print(ranger)
```

Output:

```
Name: Ranger
Color: Brown
Number of Legs: 4
Breed: Labrador
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

Adding Child-Only Methods

Child classes can also add **brand-new methods** that the parent doesn't have. These methods only exist on instances of that child.

- Dogs sometimes chase cars — but not every animal does.
- Add `chase_car` to `Dog`. `Animal` instances still don't have it.

```
class Dog(Animal):
    # ...existing code...

    def chase_car(self) -> None:
        print(f"{self.name} chases after a Honda Civic!")

ranger = Dog("Ranger", "Brown", 4, "Labrador")
ranger.chase_car()
```

Output:

```
Ranger chases after a Honda Civic!
```

WE ARE ESSENTIAL TO ALBERTA | NAIT

Guarding with isinstance()

If you have a list of mixed types and call a child-only method on every element, the program crashes when it hits a non-child item.

Use the built-in `isinstance(obj, Class)` to check the type **before** calling. It returns `True` if the object is an instance of that class (or one of its children).

```
animals: list[Animal] = [
    Dog("Ranger", "Brown", 4, "Labrador"),
    Animal("Furry Birdy", "Green", 2),
]

for animal in animals:
    animal.make_a_noise()

    if isinstance(animal, Dog):
        animal.chase_car()
```

Without the `isinstance` guard, the second iteration would crash because `Animal` has no `chase_car` method.

WE ARE ESSENTIAL TO ALBERTA | NAIT

Summary

- Inheritance lets a child class reuse a parent class's attributes and methods.
- Use `class Child(Parent):` to declare inheritance.
- Call `super().__init__( ... )` to run the parent constructor before setting child-specific attributes.
- Override a method by defining one with the same name in the child; call `super().method()` to extend instead of replace.
- Children can add new methods of their own — guard with `isinstance()` when calling them on a mixed list.

WE ARE ESSENTIAL TO ALBERTA | NAIT



Example

Let’s go run a few examples together